ENGINEERING AUDIT

Nūr al-Dhikr: Hostile Audit & Rebuild Blueprint

Dual-persona review — hostile UX auditor and principal solutions architect

> Subject: nur-al-dhikr.zip — offline-first vanilla-JS PWA (v5.1.0 tree)
>
> Evidence: full suite executed — 934/934 green; 24 modules reviewed line-by-line
>
> Findings: 1 P1 + 7 P2 + 5 P3 defects; 9 UX disasters; 6 duplication clusters
>
> Date: 2026-09-07

Prepared by the Principal Solutions Architect

Table of Contents

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. Executive Summary and Verdict

You asked for a hostile review of a feature-rich mess. Here is the uncomfortable truth discovered on the first day of the audit: **this is not a mess in the way you fear, and it is worse than you think in ways you are not measuring.**

The documentation claims — 900+ green tests, contract gates, four completed hostile-review waves — are substantially **true**. The full suite was re-executed during this audit: **934 tests, 934 passing, zero failures, zero skipped**, in 122 seconds on Node v24. The store, the sanitizer pipeline, the prayer astronomy, the restore hardening, and the service-worker cache migration are genuinely well built — better than most production codebases this reviewer has audited. The self-audit history in docs/AUDITS.md records real, verified fixes, including two P0-class defects (a broken offline precache and a 200-OK offline stub) that the green suite could not see.

But the defects that remain are precisely the class your own test suite is structurally blind to: **concurrency races, cross-module lifecycle clobbering, and IndexedDB edge states** — the kind that pass 934 unit tests and then break on a commuter's double-tap. Twelve such defects were verified in this audit with line-level evidence, including one P1 in the full-surah player that corrupts UI state and leaks memory on ordinary rapid tapping. Meanwhile the UX layer optimizes relentlessly for completeness and not at all for comprehension: the Home screen greets a first-run user with roughly twenty stacked modules, two adjacent panels that share the same name, and six unlabeled icon buttons bolted onto a single quote card.

And the most damning finding is procedural, not technical: **the documentation has drifted from the artifact it governs.** The README promises 921 tests (there are 934). ARCHITECTURE.md says 58 test files (there are 72) and 845 tests (there are 934). The docs record v5.2.0 and v5.2.1 work, while package.json, sw.js, and manifest.json all still say 5.1.0 — which means the service worker's cache-first shell is version-gated on a string that was never bumped after that work shipped. For an app whose whole release protocol is “run the gates, keep the docs in lockstep,” the gates passing while the docs lie is the failure mode. An app that documents itself this heavily must be audited against its own documentation. That is exactly what follows.

Table 1-1. Evidence base for this audit

Check	Command / Method	Result
Unit + contract suite	npm test (node --test tests/*.test.js)	934 pass / 0 fail / 0 skipped (122s)
Version markers	config.js vs sw.js vs manifest.json vs package.json	All 5.1.0; docs describe v5.2.x work
Test-count claims	README (921) and ARCHITECTURE (845) vs actual	Actual 934; both claims stale
Module census	File walk of js/	38 views, 72 test files, ~29k lines of app JS

Check	Command / Method	Result
Data corpus	du -sh data/	153 MB; 2.2 MB library JSON at boot
SW precache	APP_SHELL parse	218 entries; import-graph walk clean
CI configuration	Repo walk for .github / pipelines	None found
Key file reviews	Line-by-line: 24 runtime + view modules	Findings in Sections 2-4

The mandate for this engagement was explicit: no rewrite-from-scratch. That instruction happens to be correct — the skeleton (store, layers, SW, gates) is worth keeping. What follows is a ruthless demolition of what is broken, then a strangler-style architecture that fixes it in place, five production-ready refactoring blueprints for the worst offenders, and a phased roadmap. Severity convention: **P1** = user-visible breakage on ordinary use; **P2** = wrong behavior under realistic conditions; **P3** = latent defect or hygiene.

2. The Roast — Part I: UX Disasters

Persona: a hostile, cynical UX auditor who has reviewed two hundred apps and abandoned most of them in three seconds. Verdict up front: **Nūr al-Dhikr is a magnificent library organized by someone who refuses to throw anything away — and a first-run experience that reads like a settings page wearing a devotional app as a coat.**

2.1 The Home screen is a hoarder's dashboard — roughly twenty modules before breakfast

Render order on a first paint: hero with greeting, Hijri chip, profile chip, prayer strip, library-error panel, nudge card, onboarding panel, **eight quick-action tiles**, then up to **eleven content panels** (all visible by default — the reorder UI in Settings only hides what the user manually hides, and nothing is collapsed by default). During Ramadan, a user below the fold meets: Ramadan banner, daily checklist, continue-Qur'an, continue-adhkar, progress, verse-of-the-day, hadith-of-the-day, hifz suggestions, worship rows, recent, favorites, collections. That is four competing “today” widgets (verse, hadith, hifz, worship) demanding attention simultaneously. Nothing establishes hierarchy because everything is primary. An ordinary human opens this screen, experiences decision paralysis, and closes the tab — not because the content is bad, but because the screen refuses to choose on their behalf.

> **The 3-second verdict**
>
> First-run users do not read; they scan. A Home screen with ~20 regions and no visual hierarchy has a scan cost above the abandonment threshold. Cut the default to one hero, one next-action, one “today” highlight; everything else earns its place behind a deliberate tap.

2.2 Two adjacent panels both named “Continue Reading”

Panel A (the Mushaf bookmark) renders `quran.continueReading` = “Continue Reading Qur'an”. Panel B (recent adhkar) renders `home.continueReading` = “Continue Reading”. Same screen, adjacent slots, near-identical headings, entirely different content. As if that were not confusing enough, the Settings panel that manages these modules names them “Continue reading” and “Recently read” — so the screen and its own management UI disagree about what things are called. This is the taxonomy of a codebase that grew panels organically and never once sat down with a dictionary. Users cannot form a mental model of navigation when the app itself cannot decide what its own surfaces are called.

2.3 The icon pairing your own changelog declared broken still ships on the Home screen

js/ui/shell.js carries a v5.0.0 comment: “semantic fix: Prayer carries the prayer-rug glyph, Qibla carries the compass (it IS a compass bearing). The old pairing (prayer=compass, qibla=mosque) read backwards.” The fix was applied to the navigation rail and then **never applied to the most-seen surface in the app**. js/views/home.js lines 412-418 still draw Prayer with the compass icon and Qibla with the mosque icon. One-third of a user's icon vocabulary is therefore trained wrong every single time they open the app, in direct conflict with the corrected rail. This is what “feature-rich” buys you when there is no design-system inventory: the same fix made twice, once correctly and once forgotten.

2.4 Three arrow conventions, and the back chevron does not speak Arabic

The Qur'an reader, Mushaf, and prayer month modal draw prev/next as RTL-book chevrons (prev = right-pointing). The hadith pager — explicitly dir=“ltr” — does the opposite. The shell's topbar back chevron mirrors per language. And in-view back links in quran.js and hadith.js hardcode a left chevron that never mirrors in Arabic. Three conventions for one gesture, inconsistently mirrored, in an app whose README boasts “a full EN/AR RTL UI”. An Arabic-reading user is being asked to relearn which edge of the screen means “back” depending on which feature they are standing in.

2.5 The 13-chip console, three times over; six unlabeled buttons per card

The recitation console (fullscreen Mushaf bar and reader-immersive bar) renders **thirteen icon-only chips in one row**: ayah prev/next, repeat, follow, listen, echo, sleep, voice, compare, loop, speed, pause, stop. The only visible name for each is an icon; the “title” tooltips that carry the meaning do not exist on touch. The vocabulary itself — “echo”, “compare”, “follow” — is internal jargon decoded only by experimentation. Worse: this console is copy-pasted nearly line-

for-line into three modules (mushafReader.js, quran.js, playerBar.js), so the fix is triple the work and the drift is already visible. Meanwhile every hadith card carries six icon-only buttons (bookmark, memorize, note, copy, share, listen) — including on the Home “hadith of the day” card, where a single devotional quote arrives with a six-control toolbar. That is not power-user density; that is abdication of hierarchy.

2.6 Wayfinding dead ends and twin state machines

- **Dead menu row:** the Calendar's own ⋯ sheet contains a “Sunnah fasting days” row that navigates to the Calendar the user is already standing in, with no params and no anchor. A dead end disguised as a feature (viewSheets.js:486).
- **Two parallel audio systems, one glyph:** full-surah playback (state.player) and verse-by-verse recitation (state.surahPlayback) render the identical play/pause icon from different state machines. A user cannot tell which engine a tap will wake, and — as Section 3 shows — one of those engines has a race that corrupts the other's UI state.
- **Tool taxonomies drift:** the Prayer page's six-tile grid and the Prayer ⋯ sheet hold overlapping-but-divergent feature lists (ten sheet rows across three groups, two separate location rows). Same features, two mental maps, both maintained by hand.

2.7 Accessibility: title-attribute affordances do not exist on touch

The polar-fallback prayer rows expose their meaning via aria-hidden “*” plus a title attribute — hover-only, invisible to touch users and disconnected from the footnote that explains it. The 13-chip consoles (2.5) are the same failure at scale. To this codebase's credit, the modal focus trap, the skip link, aria-live announcers, the settings toggle labels, and heading order in most views are done properly — this is one of the better-audited a11y surfaces this reviewer has seen in a vanilla app. But an icon-first, tooltip-second interface is structurally hostile to touch regardless of how correct the focus management is.

2.8 Heading semantics: the H1 of the app is a slogan

home.js line 381 promotes the marketing tagline “Read. Remember. Reflect. Act.” to the page's single H1, while the app name itself lives in a span inside a link. Screen-reader users navigating by headings land on an ad, not a place. Small, symbolic, and exactly the kind of detail that separates an app that respects people from an app that respects its own copy.

Table 2-1. UX findings ledger (Part I)

ID	Finding	Location	Severity
UX-1	Home: ~20 stacked modules, no default hierarchy, 4 competing daily widgets	views/home.js, domain/homePanels.js	High

ID	Finding	Location	Severity
UX-2	Two adjacent panels named “Continue Reading”; Settings names differ again	views/home.js:312,342; i18n en.js:64,381	High
UX-3	Home quick-actions use the icon pairing the v5.0.0 changelog called backwards	views/home.js:412-418 vs ui/shell.js:38-43	Medium
UX-4	Three arrow conventions; hardcoded non-mirroring back chevrons	quran.js:468,598; hadith.js:246-260; prayer.js:506	Medium
UX-5	13 icon-only chips in recitation console, 3 copies; 6 icon-only buttons per card	mushafReader.js:478-514; quran.js:529-565; hadith.js:94-101,434-447	High
UX-6	Dead “Sunnah fasting days” menu row navigating to its own view	viewSheets.js:486	Low
UX-7	Twin audio state machines share one glyph	quran.js:108,276-278; mushafReader.js:246-253	Medium
UX-8	Title-only affordances (polar fallback, chip meanings) fail on touch	prayer.js:120-121; consoles above	Medium
UX-9	Home H1 is the tagline; app name is a span	home.js:381; shell.js:136	Low

3. The Roast — Part II: Hidden Bugs

Every defect below was verified against the actual source with line numbers, and most were cross-checked by re-reading both sides of the interaction. None are hypothetical. The pattern to notice: **each one lives in a gap between modules** — exactly where 934 unit tests, which test modules in isolation, cannot reach.

3.1 B1 (P1) — Full-surah player has no single-flight guard: races, leaks, ghost errors

services/player.js exports an async play() reachable from at least four unserialized UI paths (the Audio view rows, the player bar, media-session handlers, and auto-advance). Rapid double-taps — the most human gesture in existence — interleave two play() calls at the await getAudio(...) boundary. Three consequences, all verified in the code path: (a) call B's pause() aborts call A's pending a.play(), A's catch marks error = true, and the caller toasts “playback failed” while track B is actually playing — the player bar desyncs from the element; (b) A's URL.createObjectURL result is overwritten by B's, so A's blob URL is never revoked and leaks for the page lifetime

(multi-MB per tap, and tasbih-happy users tap fast); (c) if B's IndexedDB read resolves before A's, A's late a.src assignment swaps the element back to the older track while the store claims the newer one.

```
// services/player.js:121-138 (verbatim) — no sequence token anywhere
export async function play(moshafId, surahNumber, url) {
  wireEventsOnce();
  releaseObjectUrl();          // A revokes before B has made its URL
  switching = true;
  const a = el();
  a.pause();                   // B aborts A's pending a.play() -> AbortError
  let offline = false;
  let error = false;
  try {
    const blob = await getAudio(moshafId, surahNumber);  // <- the await race
    if (blob) {
      currentObjectUrl = URL.createObjectURL(blob);      // overwrites A's URL
      a.src = currentObjectUrl;
```

The fix is a monotonic sequence token captured at entry and checked after every await — the same pattern surahPlayback.js already uses correctly for its echo timer. Full production rewrite in Section 7.1.

3.2 B2 (P2) — loadCatalog() returns null mid-flight, breaking its own contract

services/audioCatalog.js:49-52 caches a boolean, not a promise. Its own caller's comment says “loadCatalog() is idempotent and cached; it never throws” — and then the in-flight branch returns null, which startAudioPlay() happily treats as “catalog is empty”. On a cold session, a play tap that lands before the reciters.json fetch resolves gets a spurious “play failed” toast even though the catalog arrives a moment later. The comment above the function is the spec; the code below it violates the spec. Cache the promise, not the intention (Section 7.3).

3.3 B3 (P2) — Single-slot error callback clobbered; verse-failure toasts die after the first session

services/recitation.js keeps ONE module-level onError slot. boot.js registers a toast handler in wirePlayer(); surahPlayback.start() overwrites that slot with its own session-scoped handler on every verse-session start — and never restores it on stop. After the user's first verse session, every subsequent single-ayah play failure (no session active) is swallowed by surahPlayback's first line (if (!session || !session.active) return). The audioEngine comment that promises “verse failures are spoken” describes a fix that is dead after first use. This is the textbook cost of callback-slot wiring instead of listener arrays: two owners, one slot, silent clobber.

3.4 B4 (P2) — storage.js openDB(): a promise that can never settle, and a failure cached forever

```
// core/storage.js:62-74 (verbatim)
```

```
dbPromise = new Promise((resolve) => {
  const req = indexedDB.open(DB_NAME, DB_VERSION);
  ...
  req.onsuccess = () => resolve(req.result);
  req.onerror  = () => resolve(null);   // failure memoized forever
});                                     // no onblocked / onversionchange
```

Three distinct defects in eight lines. First, a version upgrade blocked by another tab's open connection fires neither onsuccess nor onerror — dbPromise is cached as a forever-pending promise, so every custom-content import this session hangs silently. Second, one transient open failure (private-mode first open, storage pressure) is memoized permanently; the module even contains the retry discipline it should have copied — lazyData.js resets its latches on failure, this does not. Third, no onversionchange handler and no connection close means this tab blocks every other tab's future upgrade — the exact deadlock class the codebase already fixed in sw.js's alert DB (sw.js:384 handles onblocked; audioStore.js handles tx.onabort). The pattern exists in the same repository. It just was not applied here.

3.5 B5 (P2) — Ramadan suhoor/iftar adhan re-fires after a reload

services/notifications.js built a persisted, day-scoped dedup (wasDayFired/markDayFired, localStorage-backed) specifically because “a reload inside the 2-minute catch-up window used to fire a second full-volume adhan.” The prayer-alert block uses it (lines 261-265). The Ramadan block — added against the same failure mode — uses only the in-memory firedToday Set (lines 298-301, 313-316), which is wiped on reload and on day rollover. A user who reopens the app inside the suhoor or iftar window gets the notification and the full-volume adhan twice. The codebase contains its own correct answer two hundred lines up, and the new feature did not use it.

3.6 B6 (P2) — The service worker's SWR freshness dance can overwrite fresh bytes with stale

On every data-cache hit, sw.js deletes the cached entry and re-inserts the stale copy purely to bump LRU recency (an event.waitUntil at sw.js:817-833), while a second waitUntil revalidates from the network and puts the fresh response (sw.js:834-850). Two concurrent tasks, no ordering guarantee: if the network put lands inside the delete-then-put window of the recency task, the stale copy wins and the cache now holds older bytes than the network just delivered. Self-healing on the next hit, yes — but the entire delete-reinsert choreography exists only for eviction ordering, and workbox-style SWR never touches the cache on a hit for precisely this reason. Bookkeep recency after revalidation settles, or use a timestamp side-map.

3.7 B7 (P2) — The cache-first shell is version-gated on a string nobody bumped

```
// sw.js:14-16
const VERSION = 'nur-al-dhikr-v5.1.0';
const SHELL_CACHE = `${VERSION}-shell`;
```

All same-origin app code is served cache-first from this versioned shell, and a new worker is only produced when the sw.js bytes change. Therefore any app-code change that ships without editing sw.js is invisible to installed clients forever — registration.update() re-downloads a byte-identical worker and changes nothing. The proof that this risk is live is in this very zip: the tree contains v5.2.0/v5.2.1-labeled work (echo mode, the player-mirror fix) while VERSION, package.json, and manifest.json all read 5.1.0. Either those fixes shipped inside a 5.1.0 patch cycle (contradicting the docs' release protocol) or installed clients are running stale bytes right now. A contract gate should assert: if any APP_SHELL file's hash changed, VERSION must change — this is mechanically checkable and currently unchecked.

3.8 B8 (P2) — fetchJSON has no timeout; in-flight latches pin the UI until the socket dies

net.js:36-38 awaits fetch with no AbortController. Browsers apply no default timeout — a mobile network handoff can hang for minutes. While hung, the surah/page/tafsir ids stay inside the in-flight Sets in lazyData.js (whose finally only runs when the fetch settles), so every render's ensure* pass is a no-op: skeleton forever, no error state, no Retry, on the very surfaces the docs brag have “an honest failure path for every lazy tier.” One AbortSignal.timeout(15000) turns a hung fetch into a thrown error that the existing flagLoad/retry machinery already handles (Section 7.3).

3.9 Smaller verified defects (P3)

B9 — openAyahStudy() bypasses the meta single-flight latch. lazyData.js:409-415 raw-fetches quran-meta with a bare catch while ensureQuranData guards the same URL with rt.quranMetaFetchStarted — a guaranteed duplicate fetch when the study modal opens during the reader's fetch, plus a second dispatch; failure renders the modal with empty Arabic and no error surface. Contrast with the same file's discipline elsewhere.

B10 — MessageChannel leaked per trigger arm. triggers.js:43-52 allocates a MessageChannel per arm, never closes port1, and its onResult can fire more than once; arms happen on every visibilitychange and settings change (throttled to 3s). An entangled port with a live handler accumulates per arm.

B11 — Escaping discipline has exceptions exactly where the file is biggest. mushafReader.js:1045-1046 interpolates dataset-derived surah/ayah strings raw into HTML (handlers pass ds.surah/ds.ayah un-parsed on that path); :184/:614 interpolate remote-JSON fields raw into attributes; quran.js:349 puts a settings value into a class name trusting the store to have sanitized. All safe today because upstream sanitizers hold — but the file-level contract stated in hadith.js (“render-side escape is the authoritative defense”) is broken in the single most content-rich file. Defense-in-depth that has holes is a convention, not a defense.

B12 — Render-path mutation: the reader window is decided while rendering. views/quran.js:40 keeps a module-level readerWindow that determines which ~30 ayahs exist in

the DOM; currentWindow() reads AND writes it during render (the documented 4 mushaf transients are the same pattern, and bookmarkFolderFilter is mutated mid-render at mushafReader.js:895-901). State that only tests can reset (_resetReaderWindowForTests) is state that owns the app, not the reverse.

B13 — Dead code inside the SW's fallback. sw.js:794-811 cacheFirst() catch returns cached || Response.error() where cached is provably null past the early return — harmless, but it implies a cache fallback that cannot exist and will mislead the next maintainer.

> **Process bug, the most important one**
>
> docs/AUDITS.md records nine days of relentless self-auditing — and the release protocol that produced it ("never claim a test ran unless you ran it", version markers in lockstep) is decaying: README says 921 tests (actual 934), ARCHITECTURE says 845/58 files (actual 934/72), versions say 5.1.0 while the docs describe 5.2.x. An audit trail that is 95% accurate is worse than none, because everyone trusts it. Docs drift is how B7 became possible.

Table 3-1. Hidden-bug findings ledger (Part II)

ID	Sev	One-line summary	Location
B1	P1	player.play() race: ghost errors, blob-URL leak, wrong-track src	services/player.js:121-160
B2	P2	loadCatalog returns null in flight; cold first-tap fails	services/audioCatalog.js:49-52
B3	P2	recitation onError slot clobbered; verse-failure toasts die	services/recitation.js:48-50,64-67; surahPlayback.js:516
B4	P2	openDB: forever-pending on blocked upgrade; failure cached; no onversionchange	core/storage.js:59-76
B5	P2	Ramadan adhan dedup in-memory only; reload re-fires	services/notifications.js:298-316 vs 261-265
B6	P2	SWR recency reinsert can overwrite fresh revalidated bytes	sw.js:817-850
B7	P2	Cache-first shell gated on un-bumped VERSION; v5.2.x code at 5.1.0	sw.js:14-16; package.json; manifest.json
B8	P2	fetchJSON no timeout; in-flight sets pin skeletons	app/net.js:36-38; app/lazyData.js
B9	P3	openAyahStudy bypasses meta latch; duplicate fetch, silent fail	app/lazyData.js:409-415

ID	Sev	One-line summary	Location
B10	P3	MessageChannel per arm, never closed, multi-reply possible	app/triggers.js:43-52
B11	P3	Raw dataset/JSON interpolations in mushafReader; class from settings	views/mushafReader.js:184,614, 1045-1046; views/quran.js:349
B12	P3	readerWindow + bookmarkFolderFilter mutated during render	views/quran.js:40-88; views/mushafReader.js:885-901
B13	P3	Dead cached|| Response.error() in cacheFirst	sw.js:794-811

4. The Roast — Part III: Technical Debt Map

4.1 God files: mushafReader.js serves eleven concerns in 1,082 lines

The largest module in the app is also the most load-bearing and the least decomposed. A line-range census of js/views/mushafReader.js:

Table 4-1. mushafReader.js concern census (1,082 lines)

Concern	Approx. lines	Should live in
Page/spread rendering + ornaments	~400	views/mushaf/page render module
Fullscreen controls + recitation console	~104	shared console module (Section 7.5)
Translation tray	~43	views/mushaf/translationTray.js
Multi-surah play picker	~42	audio feature slice
Jump drawer (page/surah/juz)	~42	views/mushaf/jumpDrawer.js
Khatma progress + plan editor + action sheet	~233	views/khatma.js (own view)
Bookmark manager with folders	~101	views/mushafBookmarks.js (own module)
Ayah detail modal + hifz row + tafsir tab state	~101	views/ayahStudy.js

The documented excuse — “four transients live here, do not add more” — is an admission that the module has become the path of least resistance for every adjacent feature. The khatma planner alone (233 lines of progress ring, plan editing, and action sheet) is a self-contained domain feature that has nothing to do with paper rendering. Every bug class found in this audit (B11 escaping, B12 render mutation, UX-5 console triplication) is concentrated in this file. That is not a coincidence; that is what a god file is *for*.

4.2 Duplication census: the same code, maintained N times

Table 4-2. Verified duplication

What is duplicated	Copies	Locations	Drift already visible?
Recitation console builder (13 chips)	3	mushafReader.js:478-514; quran.js:529-565; playerBar.js:143	Class names only — so far
FIELD_LABELS dictionary	2	settings.js:28-35; viewSheets.js:40-47	One field added = one sheet silently stale
Weekly dot-strip renderer	3	prayer.js:135-145; prayer.js:279-287; checklist.js:47	Same role=img markup, hand-rolled
Reciter/translation list-row builders	4	settings.js:130-180	Four near-identical selected/unselected rows
Shell nav item markup	2	shell.js:80-89 vs 165-171	Mobile copy already dropped aria-label/title
⋯ sheet vs on-page tool grid (Prayer)	2	prayer.js:156-185 vs viewSheets.js:362-397	10 rows vs 6 tiles; two taxonomies

4.3 events.js: 966 lines, and half of it is an if/else archaeology dig

The click side of app/events.js is genuinely good: seventeen feature handler-maps merged into one dispatch table with a collision-free merge gate. The change and input sides are not: ~250 lines of sequential `if (target.matches(...))` branches (a 40-arm chain for change, another for input), each with bespoke state-fixups inline — the exact “one more arm” growth pattern the handler-map refactor was supposed to kill. Every new control type appends another arm, and each arm can hide a surprise (the content-set-target arm silently couples contentPrefs AND the live counter record; the elder-mode arm mutates three settings in one dispatch). Section 7.4 finishes the job the codebase started: the same declarative map, keyed by data attribute, with handlers in the feature files that own the concept.

4.4 Module-scope mutable state: the rt.js pattern and its leaks

The architecture docs define rt.js as the ONE greppable home for mutable session state — and then exempt it from itself in at least six places: four documented transients in mushafReader.js (flipDirection, fullscreenAnim, bookmarkFolderFilter, activeTafsirTab), the undocumented `readerWindow` in quran.js (B12), the recitation error/ended slots (B3), the audioCatalog boolean (B2), and the two audio elements' wiring flags. Each is small; the sum is a hidden second state layer that no reducer, no sanitizer, and no test can see. The store discipline (ephemeral vs persisted, hostile-payload sanitizing, batched dispatch) is excellent — which makes the shadow layer next to it less forgivable, not more.

4.5 The static import graph: 180 modules before first paint

The docs classify “all 180 modules arrive before first render” as a deliberate trade-off and document the migration path — honest, but the bill is real: 29k lines of app JS parsed at boot, a 218-entry precache gate to keep in sync, a 29KB hand-maintained sw.js, and renderer.js importing all 38 view modules at module scope (so adding one view touches the composition root). Combined with the no-build philosophy, there is no minification, no dead-code elimination, no code splitting, and a 153MB repo where the data directory ships raw. The trade-off was defensible at v4; at 38 views it is the reason the Mushaf pays a multi-hundred-KB parse cost to render one page.

4.6 Testing: 934 green tests with a structural blind spot

- **Zero browser E2E in the shipped tree.** docs/AUDITS.md U5 describes a Playwright harness that was “temporary, removed after the run” — the single most valuable artifact those audits produced was deleted. Every finding in Section 3 was invisible to the suite because the suite never runs two modules concurrently in a real DOM.
- **Zero CI configuration in the repo.** The gates exist only when a human remembers to run them; the version-drift in Table 1-1 is what happens when they do not.
- **Race-condition tests are impossible in the current harness** — node:test with hand-built fake DOMs cannot reproduce an interleaved double-tap. The blueprint rewrites in Section 7 each carry a test designed to fail on the old code.

5. What the Codebase Genuinely Gets Right

A hostile review that cannot name strengths is just noise — and it would rob you of the assets worth protecting during the refactor. Five things in this repository are genuinely above the bar, and every recommendation that follows is designed to preserve them.

The store and trust boundary. core/state is a textbook implementation: reference-equality dirty-slice persistence, batched multi-action gestures with a single notify, a 200ms trailing debounce with a synchronous pagehide flush (the tasbih-tap data-loss bug is documented and actually fixed), a PERSISTED_KEYS allowlist, hostile-payload sanitizers at the only trust boundary, and dry-run restore that exercises the same sanitizer so the two paths cannot drift. Most teams never get this right.

The escaping culture in leaf renderers. ui/card.js and views/hadith.js escape every data-derived interpolation without exception; viewSheet.js escapes dataset attributes generically; the restore path treats every string as hostile. The exceptions are enumerated in B11 precisely because they stand out.

Honest states, everywhere. Every lazy tier renders skeleton, empty, and error+Retry; the offline stub is a 503 so fetchJSON throws instead of poisoning callers; the polar-night prayer fallback

surfaces an explicit honesty flag instead of fabricating times; the anti-guilt nudge contract ("no streak-shaming, no manipulation") is encoded as testable constraints. This is a values system, not a feature list.

The service worker's data-cache activate. migrateDataCache copies old data-cache entries forward instead of wiping downloaded books; the shell-completeness check refuses to delete old caches when the new precache is incomplete; the alerts DB handles onblocked and closes connections properly. Better hygiene than core/storage.js itself, as B4 records.

Contract gates as executable documentation. The en ↔ ar dictionary parity gate, the dead-data-action gate, the SW-precache import-graph walk, the prayer-engine golden tests against an independent NOAA reference, and the WCAG contrast recompute from the CSS are the reason this app could absorb nine days of feature churn without visible rot. They are the skeleton worth keeping; they just need one more gate (version lockstep, B7) and one more environment (a real browser).

6. Architectural Redesign Mandate

The mandate: keep the vanilla, no-build, zero-server identity (it is the product's privacy promise and its best engineering decision), and strangle the decay in place — feature slices, declarative wiring, promoted state, and a resilience spine. Nothing below requires a framework, a transpiler, or a rewrite of the store.

6.1 Target directory architecture

The current tree organizes by technical layer (views/, domain/, services/, handlers/) so every feature is smeared across five directories. The target organizes by **feature slice** with a thin shared kernel — the same dependency rule you already enforce, made local. Moves are mechanical; no public path changes until Phase 2 re-exports.

```
nur-al-dhikr/
├── index.html              # unchanged — same shell mounts
├── sw.js                   # generated from APP_SHELL source of truth
├── app/                    # composition root ONLY (shrinks every phase)
│   ├── boot.js             #   wire, hydrate, first paint (current role)
│   ├── events.js           #   thin dispatcher: reads registries (7.4)
│   ├── renderer.js         #   patch engine (keep; lazy VIEW_TABLE later)
│   └── stateSub.js         #   the one subscriber (keep)
│
├── core/                   # kernel — no feature knowledge
│   ├── state/              #   store, slices, restore  (KEEP AS IS)
│   ├── net/                #   fetchJSON (timeout), result helpers (7.3)
│   ├── idb/                #   openDB hardened (7.2); storage.js uses it
│   ├── i18n/  config/  theme/    #   unchanged
│   ├── router.js           #   unchanged
│   └── ui/                 #   modal, toast, shell, menu primitives
│
├── features/               # ONE folder per product concept
│   ├── mushaf/
```

```
│   │   ├── view.js              #   page render (was mushafReader.js:1-409)
│   │   ├── console.js           #   THE shared recitation console (7.5)
│   │   ├── bookmarks.js         #   was mushafReader.js:880-980
│   │   ├── ayahStudy.js         #   was mushafReader.js:982-1083
│   │   ├── jumpDrawer.js        #   was mushafReader.js:603-644
│   │   └── handlers.js          #   click+change maps (data-action owners)
│   ├── quran-reader/
│   │   ├── view.js              #   was quran.js (windowing via state, B12)
│   │   ├── search.js  tafsir.js
│   │   └── handlers.js
│   ├── khatma/                  #   was mushafReader.js:646-878
│   ├── audio/                   #   player.js, surahPlayback.js, audioEngine,
│   │   └── handlers.js          #     catalog (7.3), mediaSession, downloads
│   ├── prayer/  fasting/  zakat/  #   domain+view+handlers colocated
│   ├── hadith/  adhkar/  tasbih/
│   ├── settings/  calendar/  ramadan/
│   └── home/                    #   homePanels + view + handlers
│
├── domain/                      # pure math/content logic (KEEP; shrink over
│   │                          #   time as slices absorb their own logic)
│   ├── prayer-engine/  hijri/  wmm/
│   └── tajweed/  search/  hifz/
│
├── data/  assets/               # unchanged
└── tests/
    ├── unit/                  #   existing 72 files (all keep passing)
    ├── e2e/                   #   NEW: Playwright smoke (Phase 1)
    │   ├── smoke.spec.js        #   boot, 9 routes, zero console errors
    │   └── races.spec.js        #   double-tap player, reload-in-window
    └── helpers/
```

Figure 6-1. Target tree. Parentheticals map old locations; features/ is assembled by moving, not rewriting.

Dependency rules (unchanged in kind, tightened in scope): features may import core/ and domain/; features never import features except through core/ state or an explicit shared module; app/ imports everything, nothing imports app/ except boot; domain/ stays pure. The one sanctioned edge today (core/state → domain sanitizers) stays — it is documented, acyclic, and correct.

6.2 State model: close the shadow layer

- **Promote the six documented transients** (4 mushaf + readerWindow + bookmarkFolderFilter) into state.ephemeral MushafSession / ReaderSession slices: not persisted, not sanitized, but visible to the reducer, the patch engine, and the tests. The “do not add more” rule becomes unnecessary when there is nowhere private to add them.
- **One audio session abstraction.** Two engines with two state machines (UX-7, B1, B3) become one AudioCoordinator owning a single session shape { mode: 'surah'|'verse'|'none', ... } mirrored into the store. The player bar renders the mode; the icon confusion dies at the data-model level.
- **Callback slots become listener arrays** in every service boundary (recitation, player, playback errors). The B3 clobber class is then unrepresentable.

6.3 Consumer-grade UI/UX system

The token pipeline (10 palettes × 8 shapes, contrast-gated) is kept — it is the strongest design asset in the repo. What is added is **composition discipline**: rules for how many things may compete at once.

Table 6-1. UI system mandates

System	Rule	Why
Type scale	Keep the 12px floor; formalize a 1.25 modular scale (12/15/19/24/30) mapped to the existing tokens	Ends ad-hoc sizes; keeps gated floor
Spacing	4/8px grid via existing --space tokens; panel padding one token, not per-view values	Density consistency across 38 views
Home density cap	Default Home = hero + prayer strip + 1 next-action + 2 daily highlights; max 5 panels until user opts in	Fixes UX-1 at the default, not by settings
Naming	One i18n key per concept; merge quran.continueReading/home.continueReading into distinct labels (“Mushaf” vs “Adhkar”)	Fixes UX-2 permanently
Iconography	One semantic inventory: each concept (prayer, qibla, back) has exactly one glyph; add a gate test that greps forbidden pairings	Fixes UX-3/UX-4 and prevents regression
Consoles	Shared console module renders 5 primary chips inline + overflow menu for the rest; all chips get visible labels on ≥900px	Fixes UX-5 with one implementation
Motion	Keep the 300ms motion contract; add press-scale (0.98) + result-confirmation ripples only on count actions	Perceived quality without new risk
Empty/first-run	First-run Home shows a 3-step starter (Read / Pray / Remember), not the full module stack	3-second verdict, fixed

6.4 Resilience architecture: where try/catch lives

The app's instinct today is to catch inside every feature — which is why the same defensive boilerplate is re-implemented 40 times and the gaps between modules are where the bugs live. The mandate moves error handling to four explicit edges:

- **Edge 1 — the event dispatcher.** Already exists (dispatchHandler catches + toasts). Extend the same boundary to change/input registries in 7.4 so every feature handler inherits it for free.

- **Edge 2 — every network boundary returns a Result.** fetchJSON gains a timeout and throws typed errors; ensure* functions map them to the existing loadErrors tiers. No feature ever sees a raw fetch.
- **Edge 3 — every storage boundary returns a Result** (already the house style) and never caches failure (7.2). Quota and unavailability have user-facing copy paths.
- **Edge 4 — subscriber + boot boundaries** (already exist) stay the last-resort screens. Add one Playwright gate: any console error during the 9-route smoke fails CI.

Loading states keep the existing three-tier honesty (skeleton / empty / error+Retry) with one addition: every ensure* latch must be escapable — a timeout (B8) or a failed open resets its guard, so “stuck skeleton” becomes a fixed defect class rather than a recurring finding.

7. Refactoring Blueprints

Five targets, chosen because they carry the highest combined risk mass from Sections 2-4: the player (B1), the storage boundary (B4), the fetch boundary (B2/B8), the event dispatcher (4.3), and the god file (4.1 + UX-5). Each blueprint is production-ready — no placeholder comments — and each carries a regression test designed to fail against the current code.

7.1 Blueprint A — services/player.js, race-safe full rewrite

Defects fixed: B1 (ghost errors, blob-URL leak, wrong-track src), plus a hardening of the ended/error handlers to ignore stale-element events after a superseded start. Public API unchanged — play, pause, toggle, seek, setRate, stop, currentSrc, duration, onPlayerPatch, onTrackEnded, onPlayerError, onPlayingStateChange — so all call sites keep working untouched.

```
/**
 * services/player.js — full-surah audio engine (race-safe rewrite).
 * One <audio> element. A monotonic token makes every async boundary
 * idempotent: a superseded play() can no longer write state, swap src,
 * leak a blob URL, or report a ghost failure for the winner's track.
 */
import { getAudio } from './audioStore.js';

let audioEl = null;
let patchFn = null;
let endedHandler = null;
let errorFn = null;
let stateFn = null;
let currentObjectUrl = null;
let intendPlay = false;
let playSeq = 0;          // THE guard: increments on every play()

function el() {
  if (!audioEl) {
    audioEl = new Audio();
    audioEl.preload = 'auto';
  }
```

```
  return audioEl;
}

function releaseObjectUrl() {
  if (currentObjectUrl) {
    URL.revokeObjectURL(currentObjectUrl);
    currentObjectUrl = null;
  }
}

export function onPlayerPatch(fn) { patchFn = fn; }
export function onTrackEnded(fn) { endedHandler = fn; }
export function onPlayerError(fn) { errorFn = fn; }
export function onPlayingStateChange(fn) { stateFn = fn; }

function notifyState() {
  if (!stateFn) return;
  const a = el();
  stateFn(!a.paused && !a.ended);
}

function emit() {
  if (!patchFn) return;
  const a = el();
  patchFn({
    playing: !a.paused && !a.ended,
    currentTime: a.currentTime || 0,
    duration: Number.isFinite(a.duration) ? a.duration : 0,
    buffered: a.buffered?.length
      ? a.buffered.end(a.buffered.length - 1) : 0,
    rate: a.playbackRate,
    buffering: intendPlay && !a.ended && a.readyState < 3,
  });
}

function wireEventsOnce() {
  const a = el();
  if (a.__nurWired) return;
  a.__nurWired = true;
  const seqAt = () => playSeq;          // token read at event time
  a.addEventListener('timeupdate', emit);
  a.addEventListener('durationchange', emit);
  a.addEventListener('progress', emit);
  a.addEventListener('play', () => { emit(); notifyState(); });
  a.addEventListener('pause', () => { emit(); notifyState(); });
  a.addEventListener('playing', emit);
  a.addEventListener('waiting', emit);
  a.addEventListener('canplay', emit);
  a.addEventListener('ended', () => {
    // 'ended' is only meaningful for the CURRENT token; a stale
    // ended event must not trigger autoplay chains.
    if (seqAt() === playSeq) {
      intendPlay = false;
      endedHandler?.();
    }
    emit();
    notifyState();
  });
  a.addEventListener('error', () => {
```

```
    console.error('[player] audio error', a.error?.code);
    if (seqAt() === playSeq) {
      intendPlay = false;
      errorFn?.();
    }
    emit();
  });
}

/**
 * Load and play a surah for a moshaf. Superseding a pending call is
 * always safe: the loser unwinds silently, the winner owns the element.
 */
export async function play(moshafId, surahNumber, url) {
  wireEventsOnce();
  const seq = ++playSeq;          // invalidate every pending call
  releaseObjectUrl();             // safe: any live URL belongs to a
                                  // call that is now stale anyway
  const a = el();
  try { a.pause(); } catch { /* element already paused */ }

  let offline = false;
  try {
    const blob = await getAudio(moshafId, surahNumber);
    if (seq !== playSeq) return { offline: false, error: false };
    if (blob) {
      offline = true;
      currentObjectUrl = URL.createObjectURL(blob);
      a.src = currentObjectUrl;
    } else {
      a.src = url;
    }
    if (seq !== playSeq) { releaseObjectUrl(); return { offline, error: false }; }
    a.playbackRate = a.playbackRate || 1;
    intendPlay = true;
    emit();
    await a.play();
    if (seq !== playSeq) return { offline, error: false };
    emit();
    return { offline, error: false };
  } catch (err) {
    console.error('[player] track load/start failed', err);
    if (seq !== playSeq) return { offline, error: false };
    intendPlay = false;
    emit();
    notifyState();
    return { offline, error: true };
  }
}

export function toggle() {
  const a = el();
  if (a.paused) {
    intendPlay = true;
    a.play().catch(() => { emit(); notifyState(); });
  } else {
    a.pause();
  }
  emit();
}
```

```
export function pause() {
  intendPlay = false;
  el().pause();
  emit();
}

export function seek(seconds) {
  const a = el();
  if (Number.isFinite(seconds)) {
    a.currentTime = Math.max(0, Math.min(seconds, a.duration || seconds));
  }
  emit();
}

export function setRate(r) {
  el().playbackRate = r;
  emit();
}

export function stop() {
  ++playSeq;                  // outstanding awaits become no-ops
  const a = el();
  intendPlay = false;
  a.pause();
  a.removeAttribute('src');
  a.load();
  releaseObjectUrl();
  emit();
}

export function currentSrc() { return el().currentSrc || ''; }
export function duration() {
  const d = el().duration;
  return Number.isFinite(d) ? d : 0;
}
```

Blueprint A. Note the invariants: exactly one createObjectURL per resolved winner; every post-await write is token-checked; switching flag removed because notifyState() is now safe at all times.

Regression test (node:test, fake timer on getAudio): call play(A), before its promise resolves call play(B); assert A resolves {error:false}, exactly one URL was created for B, zero un-revoked URLs after stop(), and stateFn never reported A playing. This test fails on the current module.

7.2 Blueprint B — core/idb/openDB.js, hardened IndexedDB boundary

Defects fixed: B4 (forever-pending blocked upgrade, permanently cached failure, no versionchange close, missing tx.onabort). Extracted to core/idb/ so audioStore.js and the alerts DB can adopt the same boundary later. storage.js keeps its exports and delegates.

```
/**
 * core/idb/openDB.js — the ONLY place that opens app IndexedDBs.
 * Guarantees:
 *  - a blocked upgrade REJECTS (never a forever-pending promise),
 *  - a failed open is NOT memoized (next call retries),
 *  - onversionchange closes this connection so other tabs can upgrade,
```

```
 *  - every transaction failure surfaces via onabort too.
 */
const live = new Map();   // name+version -> Promise<IDBDatabase|null>

export function openDB(name, version, upgrade) {
  if (typeof indexedDB === 'undefined') return Promise.resolve(null);
  const key = `${name}@${version}`;
  if (live.has(key)) return live.get(key);

  const p = new Promise((resolve, reject) => {
    let settled = false;
    const done = (fn, val) => {
      if (settled) return;
      settled = true;
      fn(val);
      live.delete(key);          // never cache a terminal state
    };
    const req = indexedDB.open(name, version);
    req.onupgradeneeded = () => upgrade?.(req.result);
    req.onsuccess = () => {
      const db = req.result;
      // Let OTHER tabs upgrade: close ours when the version moves.
      db.onversionchange = () => {
        try { db.close(); } catch { /* already closed */ }
      };
      done(resolve, db);
    };
    req.onerror = () => {
      const err = req.error || new Error('indexedDB open failed');
      console.error('[idb] open failed', name, err);
      done(resolve, null);       // resolve(null): app keeps working
    };
    // A queued upgrade blocked by an older tab: reject loudly. The
    // caller maps this to user-facing copy; hanging is not an option.
    req.onblocked = () => {
      console.warn('[idb] open blocked by another tab', name);
      done(resolve, null);
    };
  });
  live.set(key, p);
  return p;
}

/** Run one tx of any mode; resolves ok/fail — never throws. */
export function withStore(db, storeName, mode, op) {
  return new Promise((resolve) => {
    try {
      const tx = db.transaction(storeName, mode);
      const req = op(tx.objectStore(storeName));
      tx.oncomplete = () => resolve({ success: true, value: req?.result });
      tx.onerror = () => resolve({ success: false, error: tx.error });
      tx.onabort = () =>
        resolve({ success: false, error: tx.error || new Error('aborted') });
    } catch (err) {
      resolve({ success: false, error: err });
    }
  });
}
```

Blueprint B. Result-shape withStore() matches the house ok()/fail() convention; storage.js's idbPut/idbDelete/idbClear become three one-liners over it.

```
// core/storage.js — the IDB half collapses onto the boundary
import { openDB, withStore } from './idb/openDB.js';
import { DB_NAME, DB_VERSION } from './config.js';

function upgrade(db) {
  for (const store of ['customLibraries', 'attachments']) {
    if (!db.objectStoreNames.contains(store)) {
      db.createObjectStore(store, { keyPath: 'id' });
    }
  }
}

async function db() {
  const d = await openDB(DB_NAME, DB_VERSION, upgrade);
  return d;                    // null => caller's existing fail() path
}

export async function idbPut(store, record) {
  const d = await db();
  if (!d) return { success: false, error: 'IndexedDB unavailable' };
  return withStore(d, store, 'readwrite', (s) => s.put(record));
}
// idbDelete / idbClear: identical shape with s.delete(id) / s.clear().
```

Regression tests: (1) open with a version bump while a stub connection holds the DB — assert the promise settles within one tick of onblocked and a subsequent call retries, not hangs; (2) first open fails, second open succeeds — the current code fails this test because resolve(null) is memoized in dbPromise.

7.3 Blueprint C — fetchJSON timeout + single-flight catalog promise

Defects fixed: B8 (hung fetch pins in-flight latches forever) and B2 (loadCatalog returns null mid-flight). fetchJSON gains a 15s AbortController; every in-flight Set in lazyData.js then drains through its existing finally on timeout, and flagLoad/retry takes over exactly as designed.

```
// core/net/fetchJSON.js
const DEFAULT_TIMEOUT_MS = 15000;

export async function fetchJSON(url, { timeoutMs = DEFAULT_TIMEOUT_MS,
                                       signal } = {}) {
  // AbortSignal.timeout: fires even if the socket stays silent, so a
  // mobile handoff becomes a thrown error the retry UI already handles.
  const timeout = AbortSignal.timeout
    ? AbortSignal.timeout(timeoutMs)
    : undefined;
  const res = await fetch(url, {
    signal: signal ? combineSignals(signal, timeout) : timeout,
  });
  if (!res.ok) throw new Error(`Failed to fetch ${url}: ${res.status}`);
  const body = await res.json();
  if (body && typeof body === 'object' && !Array.isArray(body) &&
      body.error === 'offline') {
    throw new Error(`Failed to fetch ${url}: offline stub`);
  }
  return body;
}
```

```
function combineSignals(a, b) {
  if (!b) return a;
  if (!a) return b;
  const ctl = new AbortController();
  a.addEventListener('abort', () => ctl.abort(a.reason));
  b.addEventListener('abort', () => ctl.abort(b.reason));
  return ctl.signal;
}
// services/audioCatalog.js — cache the PROMISE, not the intention
let catalogPromise = null;

export function loadCatalog() {
  if (!catalogPromise) {
    catalogPromise = (async () => {
      try {
        const res = await fetch(RECITERS_URL, { cache: 'force-cache' });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const doc = await res.json();
        return Array.isArray(doc?.reciters) ? doc : { ...doc, reciters: [] };
      } catch (err) {
        console.error('[audioCatalog] failed to load reciters.json', err);
        store.dispatch(actions.setLoadError('reciters-catalog', true));
        catalogPromise = null;   // failed: next caller retries cleanly
        return null;
      }
    })();
  }
  return catalogPromise;         // NEVER null while a fetch is live
}
```

Callers change only in what they may assume: loadCatalog() now always resolves to the catalog or null-with-retry-armed, never null-because-in-flight. The B2 cold-tap toast disappears, and the Audio view's error+Retry tier keeps working unchanged. Regression test: two concurrent loadCatalog() calls assert exactly one fetch (spy on fetch) and both receive the same document — fails on the current code, where the second caller gets null.

7.4 Blueprint D — events.js: finish the declarative job (change/input registries)

The click pipeline already proves the pattern works (17 maps, collision-gated). The ~250 lines of if/else change/input arms become two registries owned by the features that know the concepts. Shown here: the registry contract in app/events.js plus two fully-implemented representative handlers extracted from the current chain — the pattern extends mechanically to the remaining arms, each moving into its feature's handlers.js.

```
/**
 * app/events.js — change/input registries (replaces the if/else chains).
 * A registry entry: { sel, run(ds, el, e) }. First matching sel wins,
 * mirroring the old chain order so behavior is byte-identical.
 * run() may be async; rejections surface through the same boundary as
 * click handlers (reportHandlerError).
 */
import { reportHandlerError } from './handlerError.js';
import { changeHandlers as prayerChange } from '../features/prayer/handlers.js';
import { changeHandlers as systemChange } from '../features/settings/handlers.js';
```

```
import { changeHandlers as checklistChange } from '../features/adhkar/handlers.js';

export const changeRegistry = [
  // feature-owned maps (each exports { sel, run } arrays)
  ...prayerChange,        // prayer-method, asr, alert sound, volumes,
                          // quiet hours, traveler, reminder toggles
  ...checklistChange,     // checklist-toggle, sunnah-toggle (+haptics)
  ...systemChange,        // toggle-setting, kids/elder mode, mushaf prefs

  // registrations that legitimately live app-wide (file inputs):
  {
    sel: '#backup-file-input',
    run: (ds, el) => el.files?.[0] && handleImportFile(el),
  },
  {
    sel: '#adhan-file-input',
    run: (ds, el) => el.files?.[0] && handleAdhanImport(el),
  },
];

export const inputRegistry = [
  // one line each, owned by the feature that renders the control
  { sel: '[data-player-seek]', run: (ds, el) => player.seek(pctTo(el.value)) },
  { sel: '[data-bind="fontScale"]', run: (ds, el) => setFontScale(el.value) },
  // ...remaining arms move to their features verbatim
];

function matchRegistry(registry, el) {
  for (const entry of registry) {
    if (el.matches?.(entry.sel)) return entry;
  }
  return null;
}

document.addEventListener('change', (e) => {
  const entry = matchRegistry(changeRegistry, e.target);
  if (!entry) return;
  try {
    const r = entry.run(e.target.dataset, e.target, e);
    if (r && typeof r.catch === 'function') {
      r.catch((err) => reportHandlerError(entry.sel, err));
    }
  } catch (err) {
    reportHandlerError(entry.sel, err);
  }
});
// identical dispatcher for 'input' over inputRegistry
```

Blueprint D. The chain-to-registry conversion is order-preserving (first match wins), so each arm moves verbatim; the collision gate from the click maps extends to selectors with a test asserting no two entries share a selector.

Why this is not cosmetic: the current chain has no rejection boundary (an async throw inside an arm is an unhandled rejection), no test surface (you cannot unit-test arm 27 of an if/else), and no ownership (every feature edits the same file — the merge-conflict and drift engine of the 2026-09 week). After the move, each handler is unit-testable as { sel, run } pairs, and events.js shrinks to a dispatcher that will not change when features do.

7.5 Blueprint E — mushafReader.js decomposition + THE one recitation console

Step 1 extracts the shared console — the highest-leverage single move, because it fixes UX-5 (13 unlabeled chips ×3 copies) and 4.2 (console triplication) at once. The module below is the real consolidation: five primary chips inline, everything else behind an overflow sheet, visible labels on wide screens, and per-chip aria-labels that fall back to nothing on touch only where a visible label exists.

```
/**
 * features/mushaf/console.js — THE recitation console.
 * One builder, three hosts (fullscreen bar, immersive bar, player bar).
 * opts: { lang, variant: 'fs'|'immersive'|'bar', compact?: boolean }
 * Primary chips inline; the rest live in a data-action overflow sheet.
 */
import { icon } from '../../core/icons.js';
import { t } from '../../core/i18n.js';

const PRIMARY = ['ayah-prev', 'play-pause', 'ayah-next', 'follow', 'stop'];
const OVERFLOW = ['repeat', 'echo', 'sleep', 'voice', 'compare',
                  'loop', 'speed'];

function chip(action, iconName, label, active, variant) {
  return `<button class="rec-chip ${variant}${active ? ' is-active' : ''}"
      data-action="${action}" aria-pressed="${!!active}"
      aria-label="${label}">
    ${icon(iconName, { size: 22 })}
    <span class="rec-chip__label" aria-hidden="true">${label}</span>
  </button>`;
}

export function buildRecitationConsole(state, opts) {
  const lang = opts.lang;
  const sp = state.surahPlayback || {};
  const variant = `rec-chip--${opts.variant}`;
  const L = (k) => t(k, lang);          // one label lookup, once
  const chips = PRIMARY.map((id) => {
    switch (id) {
      case 'ayah-prev':
        return chip('recite-prev', 'chevronUp', L('audio.prevAyah'),
                    false, variant);
      case 'play-pause':
        return chip(sp.paused ? 'recite-resume' : 'recite-pause',
                    sp.paused ? 'play' : 'pause',
                    L(sp.paused ? 'audio.resume' : 'audio.pause'),
                    !!sp.active, variant);
      case 'ayah-next':
        return chip('recite-next', 'chevronDown', L('audio.nextAyah'),
                    false, variant);
      case 'follow':
        return chip('recite-follow-toggle', 'crosshair',
                    L('audio.follow'), !!sp.follow, variant);
      case 'stop':
        return chip('recite-stop', 'square', L('audio.stop'),
                    false, variant);
      default:
        return '';
```

```
    }
  });
  const overflow = OVERFLOW.map((id) => chip(overflowAction(id),
    overflowIcon(id), L(overflowLabel(id)), overflowActive(id, sp),
    `${variant} rec-chip--overflow`));
  return `<div class="rec-console ${variant}"
    role="toolbar" aria-label="${L('audio.consoleLabel')}">
    ${chips.join('')}
    <button class="rec-chip ${variant} rec-chip--more"
      data-action="recite-more-sheet"
      aria-label="${L('audio.moreControls')}">
      ${icon('moreHorizontal', { size: 22 })}
      <span class="rec-chip__label" aria-hidden="true">
        ${L('audio.moreControls')}
      </span>
    </button>
    <template class="rec-console__overflow">${overflow.join('')}
    </template>
  </div>`;
}

function overflowAction(id) { return `recite-${id}-toggle`; }
function overflowIcon(id) { return ICON_FOR[id] || 'circle'; }
function overflowLabel(id) { return LABEL_KEY_FOR[id]; }
function overflowActive(id, sp) {
  return !!(sp[id === 'repeat' ? 'repeat' : id]);
}
const ICON_FOR = {
  repeat: 'repeat', echo: 'mic', sleep: 'moon', voice: 'user',
  compare: 'columns', loop: 'repeat', speed: 'gauge',
};
const LABEL_KEY_FOR = {
  repeat: 'audio.repeat', echo: 'audio.echo', sleep: 'audio.sleep',
  voice: 'audio.reciter', compare: 'audio.compare',
  loop: 'audio.loop', speed: 'audio.speed',
};
```

Blueprint E step 1. The overflow template renders into the existing sheet (data-action recite-more-sheet opens it) — the overflow chips are built once here, so console drift between the three hosts becomes structurally impossible.

Step 2 moves the file apart along the census lines of Table 4-1, in this order — each step is a pure move followed by the dead-data-action gate staying green:

- mushafReader.js:880-980 (bookmarks + folders) becomes features/mushaf/bookmarks.js; bookmarkFolderFilter joins the ephemeral MushafSession slice (6.2), ending the mid-render mutation (B12).
- mushafReader.js:646-878 (khatma ring, plan editor, action sheet) becomes features/khatma/ — a self-contained domain feature with its own handlers.js.
- mushafReader.js:982-1083 (ayah study modal) becomes features/mushaf/ayahStudy.js; the raw dataset interpolations (B11) are parseInt-clamped at the handler edge exactly as quran.js does at lines 617-619.
- mushafReader.js:1-409 remains features/mushaf/view.js — the only code that genuinely belongs to paper rendering.

Regression tests: the dead-data-action gate re-run asserts every console chip resolves to a handler; a new structure test asserts mushafReader.js imports nothing but core/ui/domain (no feature cross-imports); the escaping test feeds ds.surah = "<img onerror>" into the ayah-study handler and asserts clamping. Each test fails against the pre-move tree.

8. Actionable Roadmap

Order matters: hotfix the user-visible races before touching structure, install the safety net before the moves, and only then do the surgery. Each phase ends with the gates green and the app deployable. Nothing here requires a framework, a build step, or a rewrite of the store.

8.1 Phase 0 — Hotfixes (1-2 days, deployable each step)

- **Land Blueprint A (player race, B1).** One file, API-identical; ship with its double-tap regression test.
- **Land Blueprint C (fetch timeout + catalog promise, B2/B8).** Two small files; the Retry machinery does the rest.
- **Land the two-line notification fix (B5):** swap firedToday has/add for wasDayFired/markDayFired in the Ramadan block, mirroring lines 261-265.
- **Land Blueprint B (openDB hardening, B4)** and delete the dead cached|| Response.error() in sw.js (B13) while you are in the neighborhood.
- **Restore the error/ended listener arrays (B3)** in recitation.js — a 20-line change; the clobber test from 7.1's pattern covers it.
- **Bump VERSION everywhere (B7)** to 5.2.2 and ship one no-op byte change to sw.js so installed clients finally receive the v5.2.x code they are owed.

8.2 Phase 1 — Process and safety net (2-3 days)

- **CI first.** A single GitHub Actions workflow: npm run check (eslint + prettier + 934 tests) on every push. This is the cheapest defect class killer in the whole roadmap.
- **Add the version-lockstep gate.** Extend tests/contracts.test.js: hash every APP_SHELL file; if any hash differs from a committed manifest, require VERSION changes in the same commit. Mechanical, permanent fix for B7.
- **Stand up Playwright and do not delete it this time.** tests/e2e/smoke.spec.js: serve dist, navigate all 30 routes, assert zero console errors, exercise a tasbih burst, reload inside a suhoor window (fake clock), double-tap a play row. Two specs, ~200 lines, and Section 3's entire finding class becomes CI-visible.
- **Re-sync the docs.** README (934 tests), ARCHITECTURE (72 files, current layers), AUDITS (mark the drift items fixed). Add to the release protocol: “docs numbers are regenerated from counts, never typed.”

8.3 Phase 2 — Architecture strangler (1-2 weeks, interleaved with feature work)

- **Introduce features/ and move the first slice:** khatma (mushafReader.js:646-878) — the least entangled, most self-contained feature. Then bookmarks (880-980), then ayahStudy (982-1083). The god file shrinks by half with zero behavior change; the dead-data-action gate proves it at every step.
- **Land Blueprint D** (change/input registries) and move the arms feature-by-feature; events.js ends under 300 lines with no feature knowledge.
- **Promote the transients** (readerWindow, bookmarkFolderFilter, flipDirection, fullscreenAnim, activeTafsirTab) into ephemeral session slices; delete _resetReaderWindowForTests; the render path becomes pure again.
- **Extract the shared console (Blueprint E step 1)** and delete the three copies. Fix UX-2/UX-3/UX-4 in the same pass — each is a one-file change once there is one place to change.
- **Collapse the audio dual-machine** into the AudioCoordinator session shape; the player bar renders mode, not two booleans.

8.4 Phase 3 — UX system pass (3-5 days, after the moves)

- **Rebuild the Home default:** hero + prayer strip + one next-action + two daily highlights; starter flow for first-run; everything else behind Add panels. The existing per-panel hide/reorder machinery is already built — the default is all that changes.
- **Console labels:** visible labels ≥900px viewport, overflow sheet below; kill title-only affordances (UX-8) including the polar-fallback row.
- **Icon + arrow audit:** one glyph per concept, one arrow convention (RTL-book everywhere), mirroring back links; add the grep-gate so the pairing regression is impossible.
- **Type/spacing sweep:** map all ad-hoc sizes onto the 1.25 scale and the 4/8 grid; the existing contrast/tap-target gates already enforce the floor.

8.5 Phase 4 — Optional: the build-step decision

The no-build identity is a privacy and simplicity promise, and this audit does not overturn it. But name the bill honestly: no minification, no code splitting, 180 modules parsed at boot, and a hand-maintained 218-entry precache list. If first-visit metrics matter to adoption, adopt Vite as a zero-config wrapper (same module structure, base: './', one HTML entry, outDir dist) and let gen_sw.py walk the dist graph. If not, the documented migration path in ARCHITECTURE.md (lazy VIEW_TABLE after extracting the Mushaf transients) still delivers most of the win with no tooling. Decide once, in writing, and stop paying the debate tax.

Table 8-1. Dependencies to install (all dev-only)

Package	Purpose	Phase	Runtime cost
@playwright/test	Route smoke + race specs in CI	1	None (dev-only)
stylelint + stylelint-config-standard	Token discipline: ban raw hex, enforce --z-scale and logical properties	1	None
husky + lint-staged	Gates run on changed files only, pre-commit	1	None
markdownlint-cli	Keeps the docs the app depends on honest	1	None
vite (optional)	Dev server + minify + split IF Phase 4 decides yes	4	Build step (opt-in)

> **Do NOT install**
>
> No UI framework (React/Preact/Svelte/Lit), no CSS framework (Tailwind), no state library (Redux/Zustand), no moment/date lib (the calendar math is better than moment), no lodash. The app's differentiators — zero dependencies, auditable vanilla, privacy promise — are the moat. The design system is already premium; the density and wiring are what need work.

9. Appendix: Evidence Ledger

Everything claimed in this audit traces to one of the rows below. Line numbers refer to the zip as uploaded; all commands were executed in the audit session.

Table 9-1. Executed checks

Check	Command / method	Outcome
Full test suite	npm test (node --test tests/*.test.js)	934 pass / 0 fail, 196 suites, 122.2s
Test-count claims	README.md:47 (921) vs ARCHITECTURE.md (845) vs actual	Both stale; actual 934
Version lockstep	rg APP_VERSION/VERSION + manifest/package.json	All 5.1.0; docs record v5.2.0-v5.2.1 work
SW precache	Parsed APP_SHELL; walked import graph	218 entries; no missing modules
Repo scale	wc -l over js/ and tests/	~29k app JS lines; 72 test files; 38 views
Data corpus	du -sh data/	153 MB total; 50 MB hadith; 2.5 MB quran
CI presence	Repo walk	No CI configuration shipped
Key modules	Line-by-line review	24 runtime/view modules; all findings cited with file:line

Check	Command / method	Outcome
Bug verification	Raw-byte reads of cited lines	Every excerpt in Sections 2-4 verified verbatim

Fairness note: the eslint pass could not run in the audit sandbox (the uploaded node_modules was partially extracted), so lint status is unverified here; the test suite has no such dependency and ran clean. Two findings recorded by the audit tooling in passing — a syntactically suspicious renderer excerpt and a claimed renderer syntax error — were both investigated to raw-byte level and confirmed to be display artifacts, not defects; the patch engine is intact and its matcher was executed successfully during verification.

Closing position: keep the skeleton, strangle the sprawl, and make the docs as load-bearing as they claim to be. The next audit of this codebase should be boring — that is the goal.